Algorithm:

Input: User Query Q, Confidence Threshold τ (e.g., 70)
Output: Final Factual Answer A*
Initialize: trusted_done = False, general_done = False

1: A_init <- Generate(Q) *(Zero-shot baseline)*
2: C <- ExtractClaims(A_init)
3: V <- CheckConstraints(C, Q)
4: if V contains no constraint violations then
    5: S_intrinsic <- ScoreIntrinsic(C) *(Closed-book evaluation)*
    6: if S_intrinsic >= τ then
        7: return A_init *(Bypass retrieval, exit early)*
    8: end if
9: end if
10: Domain <- DetectDomain(Q)
11: while True do
(1. Adaptive Search Routing)
    12: if not trusted_done then
        13: D_raw <- TrustedSearch(Q, Domain)
        14: trusted_done <- True
    15: else if not general_done then
        16: D_raw <- GeneralWebSearch(Q)
        17: general_done <- True
    18: else
        19: return GenerateApology() *(Circuit breaker triggered)*
    20: end if
(2. Corrective Document Grading (CRAG))
    21: D_filtered <- GradeDocuments(D_raw, Q)
    22: if D_filtered is empty then
        23: continue *(Loop to next search tier)*
    24: end if
    (3. Extrinsic Regeneration & Verification)
    25: A_regen <- Regenerate(Q, D_filtered)
    26: C_regen <- ExtractClaims(A_regen)
    27: S_retrieved <- ScoreRetrieved(C_regen, D_filtered)
    28: if S_retrieved >= τ then
        29: return A_regen *(Verified RAG answer)*
    30: else if trusted_done and general_done then
        31: return GenerateApology() *(Exhausted all resources)*
    32: end if
    33: end while

This algorithm represents a highly optimized, "fail-safe" retrieval pipeline. At its core, it is designed to solve two massive problems in modern AI: **hallucinations** and **wasted compute costs**.

Instead of blindly searching the web for every question (which is expensive and slow), it acts like a human test-taker. First, it tries to answer from memory (closed-book). If it is absolutely certain it knows the answer, it stops. If it is uncertain or fails a strict constraint check, it opens the book (searches the web), but it does so in stages—checking trusted sources before falling back to the open web. If it still can't find the truth, it gracefully admits defeat rather than guessing.

Here is a complete breakdown of the algorithmic flow and the underlying functions.

Phase 1: The "Closed-Book" Attempt (Lines 1-9)

The system attempts to answer the query instantly using its internal parametric memory to save time and API costs.

- **Step 1:** Generate a zero-shot answer.
- **Step 2-3:** Break the answer into verifiable facts and check if they violate any rules (e.g., the user asked for events in 1987, but the answer mentions 1990).
- **Step 4-8:** If the rules are respected, a strict Critic evaluates how confident the model is in its own facts. If the confidence score ($S_{intrinsic}$) meets the threshold (τ), the system halts and outputs the answer immediately.

Phase 2: Adaptive Search Routing (Lines 10-21)

If the closed-book attempt fails or is uncertain, the system triggers the external retrieval loop.

- **Step 10:** It detects the specific domain of the query (e.g., medical, legal, sports).
- **Step 12-18:** It uses a tiered approach. It first tries a `TrustedSearch` (e.g., searching strictly verified medical databases). If that fails to yield a good answer later in the loop, the `while` loop restarts, and it broadens its scope to a `GeneralWebSearch`.
- **Step 19-21:** If both search tiers have been exhausted, the circuit breaker trips. The system realizes the information does not exist online and issues an apology.

Phase 3: Corrective Document Grading (Lines 22-26)

This is the "CRAG" (Corrective Retrieval-Augmented Generation) step.

- **Step 23:** Before letting the generator see the search results, it evaluates the raw documents for relevance.
- **Step 24-26:** If the search engine returned garbage or irrelevant links, the system immediately abandons this search tier and loops back to try the next one.

Phase 4: "Open-Book" Regeneration & Verification (Lines 27-36)

- **Step 28:** The generator gets a second chance, but this time it is strictly instructed to use *only* the filtered documents to write a concise answer.
- **Step 29-30:** The new answer is broken into facts, and the Extrinsic Critic checks if every single fact is explicitly supported by the retrieved text.
- **Step 31-36:** If the grounded score passes the threshold (τ), the verified answer is returned. If it fails (e.g., the generator hallucinated again), it loops back, or issues an apology if all search options are exhausted.

Phase 1: Generation & Atomic Decomposition

This phase attempts to answer the query efficiently using parametric memory and prepares the text for rigorous mathematical evaluation.

- **Generate(Q)**
  - **Mechanism:** This is the baseline, zero-shot inference pass. The LLM receives the user query Q and generates an initial answer A_{init} relying purely on the knowledge embedded in its pre-trained weights.
  - **Architectural Purpose:** It serves as the primary compute-saving mechanism. If the model already knows an established, universal fact, bypassing the retrieval step drastically reduces latency and API costs.
- **ExtractClaims(Text)**
  - **Mechanism:** Inspired by the FActScore methodology, this function acts as a linguistic annotator. It takes a complex, multi-sentence response and decomposes it into an array of independent, atomic claims C. It performs **coreference resolution** (e.g., replacing pronouns like "He" with explicit entities like "Attaphol Buspakom") to ensure each claim can be verified in a vacuum.
  - **Architectural Purpose:** Standard LLM-as-a-judge prompts struggle to evaluate long paragraphs accurately. By forcing the text into atomic units, the downstream evaluators can pinpoint exactly which specific sentence contains a hallucination without failing the entire response.

Phase 2: The Intrinsic (Zero-Shot) Filter

This phase ruthlessly tests the baseline answer to see if it is safe to show the user, or if it requires external verification.

- **CheckConstraints(C, Q)**
  - **Mechanism:** A deterministic validation layer. It analyzes the query Q for explicit boundaries (e.g., Temporal: "Between 1987 and 1988"; Exclusionary: "Do not mention X"). It then checks if any claim in C violates these rules.
  - **Architectural Purpose:** Prevents "technically true but contextually wrong" hallucinations. If a model provides a historically accurate fact that falls outside the user's requested timeframe, this function flags it as a violation, forcing a regeneration.
- **ScoreIntrinsic(C)**
  - **Mechanism:** A strict, binary (0.0 or 1.0) closed-book evaluator. It grades the extracted claims based purely on the LLM's confidence in its parametric knowledge.
  - **Architectural Purpose:** This is the gatekeeper. If the model generates a highly specific statistic, exact date, or niche entity, this function is instructed to distrust it and score it 0.0. This intentionally tanks the overall score below the threshold τ, successfully forcing the system to trigger the RAG retrieval loop.

Phase 3: Adaptive Retrieval Routing

If the intrinsic memory is insufficient, the system intelligently searches for external data.

- **DetectDomain(Q)**
  - **Mechanism:** A lightweight classification routing function. It reads the query and assigns a domain label (e.g., Medical, Legal, Academic, Sports).
  - **Architectural Purpose:** Narrows the search space. Searching the open web for

legal advice often yields poor SEO-optimized blogs. Knowing the domain allows the system to route to specialized databases first.

- **TrustedSearch(Q, Domain)**
  - **Mechanism:** A high-precision, low-recall retrieval function. It queries strictly curated, verified databases (e.g., PubMed, official Wikipedia APIs, or proprietary vector stores) corresponding to the detected domain.
  - **Architectural Purpose:** Prioritizes authoritative truth over broad, potentially poisoned web data.
- **GeneralWebSearch(Q)**
  - **Mechanism:** The fallback, high-recall retrieval function. If the trusted search returns nothing, it queries the open web (e.g., via Tavily or Google Search APIs) to scrape broader information.

Phase 4: Context Processing & Extrinsic Verification

This phase filters the scraped data, rewrites the answer, and performs the final grading.

- **GradeDocuments(D_raw, Q)**
  - **Mechanism:** The core of the Corrective-RAG (CRAG) framework. This function evaluates every retrieved document chunk in D_{raw} against the query Q. If a chunk is semantically irrelevant, it is explicitly deleted.
  - **Architectural Purpose:** Mitigates **Context Poisoning**. Feeding an LLM irrelevant or contradictory search results is a primary cause of RAG hallucinations. This function acts as a strict bouncer, ensuring the generator only sees highly relevant text $D_{filtered}$.
- **Regenerate(Q, D_filtered)**
  - **Mechanism:** A highly constrained, open-book generation pass. The LLM is instructed to be ruthlessly concise, literal, and to base its answer *exclusively* on $D_{filtered}$, appending source URLs to its claims.
  - **Architectural Purpose:** Solves the "chattiness" liability. By preventing the model from adding unprompted background trivia, it drastically reduces the surface area for accidental extrinsic hallucinations.
- **ScoreRetrieved(C, D_filtered)**
  - **Mechanism:** The nuanced Extrinsic Critic. It verifies if the regenerated claims are logically entailed by the retrieved text using a 3-tier grading system (Correct, Partial, Incorrect).
  - **Architectural Purpose:** Unlike the ruthless Intrinsic scorer, this function uses logical deduction. The "Partial" label allows the system to recognize **Temporal Entailment** (e.g., understanding that the specific month "April 1987" is logically supported if the text confirms the years "1985 to 1989"), preventing the system from unfairly rejecting a heavily grounded answer due to a minor precision mismatch.
- **GenerateApology()**
  - **Mechanism:** The terminal circuit breaker. If all search tiers are exhausted and the regenerated answers still fail to meet the τ threshold, this function outputs a standardized, graceful refusal (e.g., "I cannot confidently answer this query based on available sources.").
  - **Architectural Purpose:** Enforces the ultimate rule of factual QA: *It is always better to admit ignorance than to hallucinate a false truth.*
